Assignment - 01

Title : Searching and Sorting

Objective : To organize the data and extract data based on given Condition.

Problem Statement :
Consider a student database of SEIT class (at least 15 records). Database contain different field of every student like Roll no, name and Sgpa (array of struc)
a). Design a roll call list, arrange list of student acc. to roll no. in ascending order (use Bubble Sort)
b). Arrange list of student alphabetically (Insertion Sort)
c). Arrange list of student to find out 1st ten toppers from a class (use Quick Sort)
d). Search student acc. to SGPA. If more than one student having same SGPA, then print list of all the student having same SGPA.
e). Search a particular student acc. to name using binary search without recursion (all the student record having the presence of search key should be displaye).

Outcome : Analyze algorithm and to determine algorithm correctness and time efficiency class.

Theory :

1. Bubble Sort ;- It is the simplest sorting algorithm that work by repeatedly swapping adjacent element if they are in wrong order.
   - In this, only 2 element can be swapped at a time.

e.g ;- Sort the given element in ascending order [15, 10, 8, 2, 3, 7, 11]

First pass: 15, 10, 8, 2, 3, 7, 11
→ 10, 15, 8, 2, 3, 7, 11
Swap (15 & 10)
→ 10, 8, 15, 2, 3, 7, 11
Swap (8 & 15)
→ 10, 8, 2, 15, 3, 7, 11
Swap (15 with 2)
→ 10, 8, 2, 3, 15, 7, 11
Swap (3 & 15)
→ 10, 8, 2, 3, 7, 15, 11
Swap (7 & 15)
→ 10, 8, 2, 3, 7, 11, 15
Swap (11 & 15)

Second pass : 10, 8, 2, 3, 7, 11, 15
→ 8, 10, 2, 3, 7, 11, 15 (Swap 8 & 10)
→ 8, 2, 10, 3, 7, 11, 15
→ 8, 2, 3, 10, 7, 11, 15
→ 8, 2, 3, 7, 10, 11, 15

Third pass : 8, 2, 3, 7, 10, 11, 15

2, 8, 3, 7, 10, 11, 15

2, 3, 8, 7, 10, 11, 15

2, 3, 7, 8, 10, 11, 15

Fourth pass : 2, 3, 7, 8, 10, 11, 15

2, 3, 7, 8, 10, 11, 15

∴ Sorted array = 2, 3, 7, 8, 10, 11, 15.

- Algorithm :

1. Read the total no. of element's say n.
2. Store the element in the array.
3. Set the i = 0.
4. Compare the adjacent elements.
5. Repeat step 4 for all n elements.
6. Increment the value of i by 1 and repeat step 4, 5 for i<n.
7. print the sorted list of element.
8. Stop.

2. Insertion Sort : In this, the element are inserted at their appropriate place.

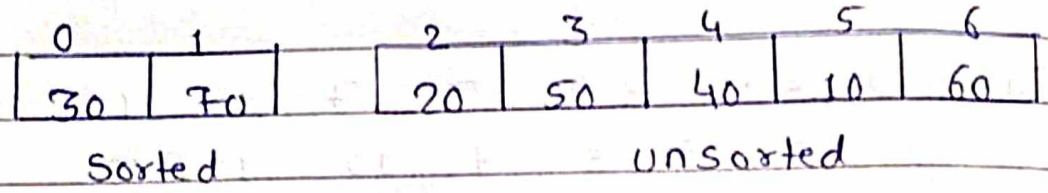
e.g :-

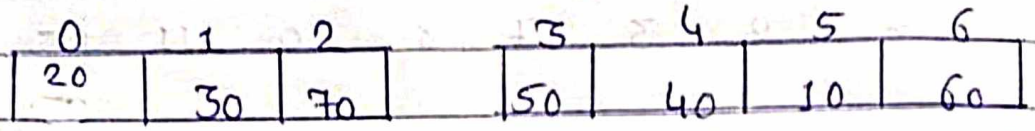
0	1	2	3	4	5	6
30	70	20	50	40	10	60

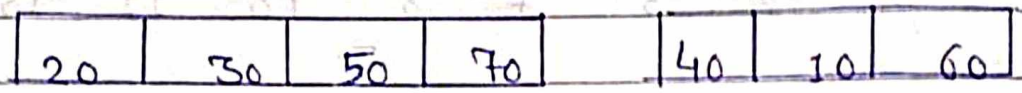
The process start with first element.

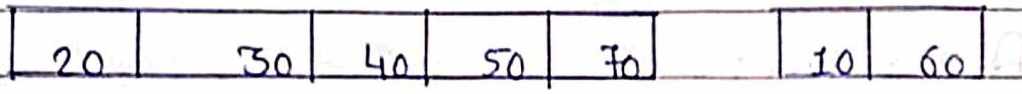
0							
30	70	20	50	40	10	60	Compare 70

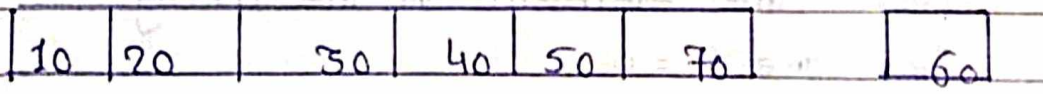
Sorted | Unsorted | With 30 and insert it at it's position.

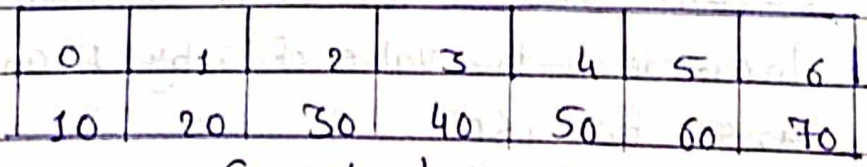












Sorted list of element

- Algorithm :-

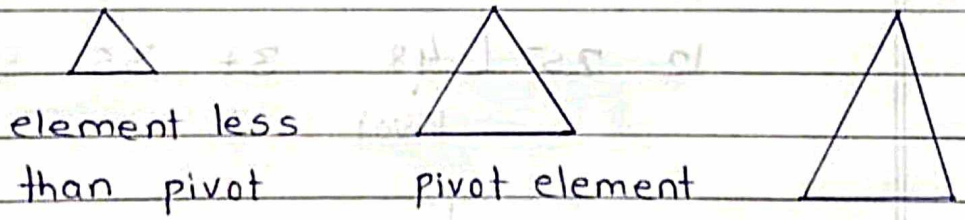
```
Insert-Sort (A[0..n-1])
    Input : An array of n element's
    Output : Sorted array A[0..n-1] in
             ascending order.
For i ← 1 to n-1 do
{
    temp ← A[i] // mark A[i]th element
    j ← i-1 // set at previous element of A[i]
While (j >= 0) AND (A[j] > temp) do
```

```
        A[j+1] ← A[j]
        j ← j-1
    }
    A[j+1] ← temp // Copy A[i] element at A[j+1]}
```

3. Quick Sort : It is a Sorting algorithm that use the divide and conqure strategy. In this Method division is dynamically carried out.

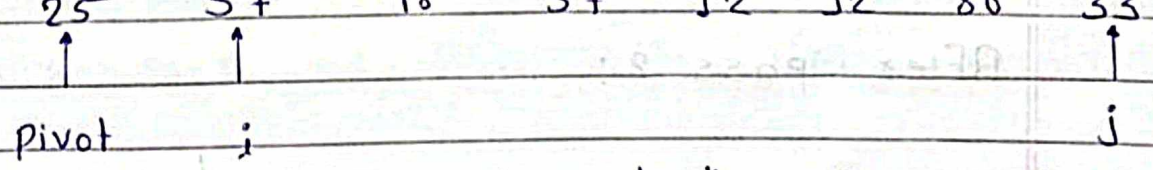
Divide : Split the array into two sub array that each element in the left sub array is less than or equal to the middle element in the right sub array is greater than the middle element.

Pivot element ; The splitting of the array into two sub array is based on pivot element.



e.g;- 25, 57, 48, 37, 12, 92, 86, 33

Consider the first element as a pivot element



Now, if A[i] < pivot element then increment i. And if A[j] > pivot element then decrement j. When we get these above Condition to be false Swap A[i] and A[j].

25 33 48 37 12 92 86 57

pivot i j

25 33 48 37 12 92 86 57

pivot i j

Swap A[i] and A[j]

25 12 48 37 33 92 86 57

pivot j i

As j > i swap pivot with A[j]

∴ After pass 1:

12 25 48 37 33 92 86 57

Now, Consider 48 to be as pivot

12 25 [48 37 33 92 86 57]

pivot i j

12 25 [48 37 33 92 86 57]

pivot i j

12 25 [48 37 33 92 86 57]

j i

j > i, Swap A[j] with pivot 48

After pass 2:

12 25 33 37 48 [92 86 57]

Pass 3 ;-

12 25 33 37 48 92 86 57

Assume 92 as a pivot

12 25 33 37 48 92 86 57

Pivot i j

12 25 33 37 48 92 86 57

pivot j i

j > i Swap A[j] With pivot

12 25 33 37 48 57 86 92

Pass 4;-

12 25 33 37 48 57 [86 92]

in pass 5 ;- [After pass 5]

12 25 33 37 48 57 86 92

is a sorted list.

Sorting Method	Best Case	Avg Case	Worst case
1. Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
2. Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
3. Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

4. Linear Search :- Searching technique in which every element is compared with key linearly from First to last element of list. The time Complexity of this algorithm is O(n). The time Complexity will increase linearly with the value of n.

- Advantage's of linear Searching
  1. It doe's not require specific ordering before applying the Method.
- Dis-advantage of linear Searching
  1. It is less efficient as Compared to Binary Search.

e.g:-

Input arr[ ] = { 10, 20, 80, 30, 60, 50, 110, 100 }
x = 110;

Output : 6

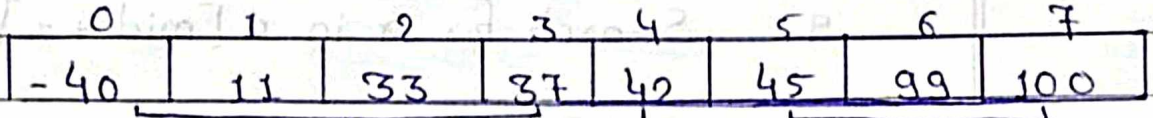
and if we put x = 1000 then our output : -1

- Start from the left-most element of arr[ ] and one by one Compare x With each element of arr[ ]
- If x matches With an element, return the index.
- If x doesn't Match With any of element return -1.

5. Binary Search : Searching algorithm in which the list of element is divided into two sublist and the key element is compared with the middle element. If the match is found then, the location of middle element is returned otherwise, we search into either of the halves (sublist) depending upon the result produced through the match.

The prerequisite for this searching technique is that the list should be sorted.

e.g :-



Step 1 :- The key element which is to be Searched is 99 ∴ key = 99.

Step 2 :- Find the middle element of the array. Compare it with the key.

if mid ? key

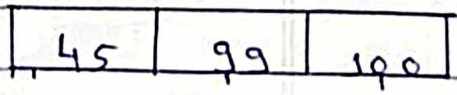
i.e 42 = 99 no,

if 42 < 99 search the sublist 2.

Now, we only Have to Search the Sublist 2.

again divide the sublist 2 & find mid element

Now Mid = 99 check,



if mid = key

99 = 99 yes.

So, Match is found at 7th position of our given array.

Algorithm for Binary Search :-

```
1. if (low > high)
2. return;
3. mid = (low + high) / 2;
4.    if (x == a [mid])
5.    return (mid);
6.    if (x < a [mid])
7.    Search for x in a[low] to a [mid - 1];
8.    else
9.    Search for x in a [mid + 1] to a [high];
```

Searching Method	Time Complexity
Linear Search	O(n)
Binary Search	O(n log n)

Test Case's :-

Test Case	Test Case Description	Test	Expected Result	Actual Result	pass/ fail
1.	Check result When User select option Bubble Sort	Student Roll No.	All data arranged in ascending order	All data is arrang in ascen- ding roll wise	pass.
2.	Check result When user Select option insertion sort	Student Name	All data get arrange in ascen. alpha- betically	All data is in ascending alphabetically	pass
3.	Check result When User select option Quick sort	Student SGPA	All data is arranged descending order	All data is arranged in descend- ing order	pass
4.	Check result When User Select search option.	Student SGPA	User input is Compared With data & Ma- tch data get displayed	User input Matched data get displayed	Pass
5.	Check result When User select Binary Search	Student Name	User input Compered With name data & Matched data display	User input With name data is displayed	pass

```
#include <iostream>
#include <cstdio>
#include <iomanip>
#include <string.h>


using namespace std;


struct student{
   unsigned int roll_no;
   char name[50];
   float sgpa;
};
```

```
class List{
   private:
   student a[20];
   int size;
   public:

   int get_size(){
      return size;
   }

   // This is create method
   void create(){
      cout << "Enter the size of Array :";
      cin >> size;  // Takes the size and input
      for (int i=0;i<size;i++){
      cout << "Enter the Roll No. ";
      cin >> a[i].roll_no;
      cout << "Enter the Name :";
      cin >> a[i].name;
      cout << "Enter SGPA :";
      cin >> a[i].sgpa;

      }
   }

   // This is bubble_sort to sort according to roll_no
   void bubble_sort(){
    for (int i=0;(i < size);i++){
      for (int j=0;j<size-1;j++){
```

```
        if (a[j].roll_no>a[i].roll_no){
        student temp = a[i];
        a[i] = a[j];
        a[j] = temp;
        }
      }
    }


    cout << "Roll No." << setw(15) << "Name" << setw(25) << "SGPA"<< endl;
    for (int i=0;i<size;i++){

      cout << a[i].roll_no << setw(25) << a[i].name << setw(30) << a[i].sgpa <<endl;
    }
  }

// This is insertion sort to sort according to name
void insertion()
{
   int i, j;
   student key;
   for (i = 1; i < size; i++)
   {
      key = a[i];
      j = i - 1;

      while (j >= 0 && a[j].name > key.name)
      {
         a[j + 1] = a[j];
```

```
            j = j - 1;
        }
        a[j + 1] = key;
    }

    cout << "Roll No." << setw(15) << "Name" << setw(25) << "SGPA"<< endl;
    for (int i=0;i<size;i++){

        cout << a[i].roll_no << setw(25) << a[i].name << setw(30) << a[i].sgpa <<endl;
    }
}

// this is normal function to find freq of highest marks
void find_freq(){
        int freq;
        float freq_marks;
        int max_freq = 1;
        for (int i=0;i<size;i++){
            int freq =1;
            for (int j=0;j<size;j++){
                if (a[j].sgpa == a[i].sgpa){
                    freq++;
                }
            }
            if (max_freq<=freq){
                max_freq = freq;
                freq_marks = a[i].sgpa;
            }
        }
```

```
      cout << "The maximum frequency of the marks is : "<< max_freq-1  << " Which is marks" << freq_marks << endl;
   }

// This is linear_search to find students according to SGPA
void linear_search(int find){
   for (int i=0;i<size;i++){
      if (a[i].sgpa == find){
         cout << "Roll NO:-" << a[i].roll_no << endl;
         cout << "Name :- " << a[i].name << endl;
         cout << "SGPA :-" << a[i].sgpa << endl;
      }
   }
}

// Quick sort

   int partation(int low,int high){
      student pviot = a[low];
      int i = low+1;
      int j = high;

      do{
      while(a[i].sgpa >= pviot.sgpa){
         i++;
      }

      while(a[j].sgpa < pviot.sgpa){
         j--;
```

```
    }

    if (i<j){
    student temp = a[j];
    a[j] = a[i];
    a[i] = temp;
    }
    }while(i<j);

    student temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j;
}

void quick_sort(int low,int high){
    if (low<high){
        int part = partation(low,high);

        quick_sort(low,part-1);
        quick_sort(part+1,high);
    }
}

// this method uses quick_sort to sort 1st 10 SGPA
void display_top(){
    quick_sort(0,size-1);
    if (size>10){
```

```
    cout << "Roll No." << setw(15) << "Name" << setw(25) << "SGPA"<< endl;
     for (int i=0;i<10;i++){
        cout << a[i].roll_no << setw(25) << a[i].name << setw(30) << a[i].sgpa <<endl;
     }
  }
  else{
     cout << "Roll No." << setw(15) << "Name" << setw(25) << "SGPA"<< endl;
       for (int i=0;i<size;i++){
        cout << a[i].roll_no << setw(25) << a[i].name << setw(30) << a[i].sgpa <<endl;
       }
    cout << "Not enough students" << endl;
  }
}



void search_student(){
  char find[50];
  cout << "Enter the name :";
  cin>>find;

  for (int i=0;i<size;i++){
    if (strcmp(a[i].name,find)==0){
       cout << "Name : " << a[i].name << endl;
       cout << "Roll No : " << a[i].name << endl;
       cout << "SGPA : " << a[i].sgpa << endl;
    }
  }
}
```

```
};

int main()
{

   List s;
   s.create();
   bool flag = true;
   while (flag){

   int ch, i;
   cout << "\n 1: Arrange list according to roll no.(bubble sort)";
   cout << "\n 2: Arrange list of students alphabetically. (Use Insertion sort)";
   cout << "\n 3: Search students according to SGPA. If more than one student having same SGPA, then print list";
   cout << "\n 4: Most appeared marks \n";
   cout << "\n 5: Search Student According to name \n";
   cout << "\n 6: Exit \n";
   cout << "\n Enter your choice : ";
   cin >> ch;

   switch(ch){
```

```
        case 1:
        s.bubble_sort();
        continue;
        case 2:
        s.insertion();
        continue;
        case 3:
        s.display_top();
        continue;
        case 4:
        s.find_freq();
        continue;
        case 5:
        s.search_student();
        continue;
        case 6:
        flag = false;
        default:
        cout << "Invalid";
    }
    }



    return 0;
}
```